Programa:	GoLive_CS	
Protección:	Empaquetado con Vbox y Trial 30 días	
Objetivos:	Desempaquetar el archivo (el Trial cae directo al desempakar, jeje..)	
URL descarga:	---	
Dificultad:	Sabiendo lo que es la IAT no problema… ;DDD	
Herramientas:	OllyDbg + PEiD + LordPE + ImpRec.	
Cracker:	SyXe'05 [hAcK-c0d3d]	Fecha: 28-07-2005

Hola a todos otra vez..!!

Bueno pues estamos de nuevo con otro Vbox y también de la compañía Adobe, en este caso ***GoLive CS*** y lo desempakaremos con el mismo método que para Illustrator 10 del anterior tute de esta misma compañía y protección, conocedores de estos pasos podemos avanzar rápido pero me detendré bastante por si alguien se acaba de incorporar no quiero que nadie se me quede atrás OK?

Empezamos…

Bien como todo buen Cracker ¿? Abrimos PEiD para saber con qué está empaquetado, nosotros ya lo sabemos gracias a la introducción así que obviaremos este paso, para más información:

Packer: ***VBOX 4.3 - 4.6.x -> WeijunLi***

OK? Sigamos a lo nuestro, como nos queda tiempo del Trial todavía podemos acceder al momento exacto en que se pone a rellenar la IAT, pues hay algunos Vbox que tienen mágico y hay otros que no lo tienen (almenos que yo sepa) y hay que injertar código, para salir de dudas cargamos el programa en ***Olly***, más concretamente el ejecutable ***GoLive.exe***, aparece el típico mensaje diciendo que el EP está fuera de la sección ***.code***, aceptamos y aterrizamos en el EP si no queremos que Olly lo analice le damos a la barra espaciadora, ahora pulsamos ***F9*** para que corra el programa, nos aparecen tres mensaje s como el inicial para vboxat.dll, vboxtb.dll y site.dll, o sea que estas librerías están empaquetadas también, jeje.. no ves como se las gasta no? Aceptamos los tres mensajes y el programa carga, bien no tiene antidebug es bueno saberlo, reiniciamos con ***Ctrl+F2*** y aceptamos los mensajes, sale la nag inicial:

esta claro que se trata de Vbox no? Jeje.. como para no verlo vaya, pero nosotros a lo nuestro ahora vamos a ***Olly*** nos metemos en la ventana de memoria con ***Alt+M*** y ponemos un ***BPM*** en la primera sección para que pare allí,

hecho esto aceptamos la nag picando en el botón ***Try*** y automáticamente caemos de vuelta en Olly, veamos donde:

```
File View Debug Plugins Options Window Help
07028210   8807          MOV BYTE PTR DS:[EDI],AL
07028212   8A46 01       MOV AL,BYTE PTR DS:[ESI+1]
07028215   C1E9 02       SHR ECX,2
07028218   8847 01       MOV BYTE PTR DS:[EDI+1],AL
0702821B   83C6 02       ADD ESI,2
0702821E   83C7 02       ADD EDI,2
07028221   83F9 08       CMP ECX,8
07028224 ^ 72 A6         JB SHORT vboxtb.070281CC
07028226   F3:A5         REP MOVS DWORD PTR ES:[EDI],DWORD PTR D
```

veamos en la barra de estado porqué paro:

```
Memory breakpoint when writing to [008E548E]
```

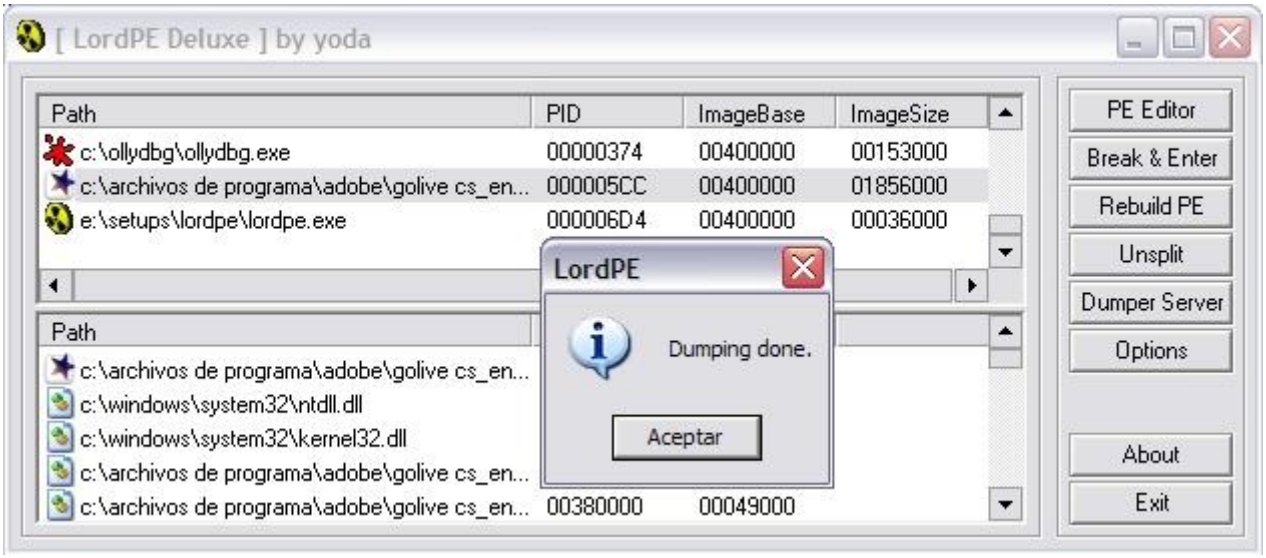
Paro por un acceso de ***escritura*** en la sección donde tenemos el ***BPM*** puesto, como no indica nada esto le damos a ***F9*** nuevamente, asi dos o tres veces y mirad donde caemos:

```
OllyDbg - GoLive.exe - [CPU - main thread, module GoLive]
File View Debug Plugins Options Window Help
008E548E   6A 74          PUSH 74
008E5490   68 9067A000    PUSH GoLive.00A06790
008E5495   E8 66020000    CALL GoLive.008E5700
008E549A   33DB           XOR EBX,EBX
008E549C   895D E0        MOV DWORD PTR SS:[EBP-20],EBX
008E549F   53             PUSH EBX
008E54A0   8B3D 642D9600  MOV EDI,DWORD PTR DS:[962D64]     kernel32.GetModuleHandleA
008E54A6   FFD7           CALL EDI
008E54A8   66:8138 4D5A   CMP WORD PTR DS:[EAX],5A4D
008E54AD v 75 1F          JNZ SHORT GoLive.008E54CE
008E54AF   8B48 3C        MOV ECX,DWORD PTR DS:[EAX+3C]
008E54B2   03C8           ADD ECX,EAX
008E54B4   8139 50450000  CMP DWORD PTR DS:[ECX],4550
008E54BA v 75 12          JNZ SHORT GoLive.008E54CE
```

No es un ***PUSH EBP*** pero es el ***OEP*** ya que las API's que lo rodean son más que sospechosas, se ejecutan justamente las que se suelen utilizar al inicio de casi todos los programas y en ese mismo orden, muy probablemente esté compilado en ***Delphi*** de ahí el primer call a ***GetModulehandleA*** como siempre hace en el primer call, aquí lo hace pero sin entrar en ningún call, pero bueno ya tenemos el primer dato:

OEP: 8E548E

Este es un buen momento para dumpear el archivo, abramos LordPE activar el ***intellidump*** y hacer un ***dump full***, le damos un nombre y listos, la cosa se vera asi cuando hayamos hecho esto:



Aceptamos el mensaje y ya tenemos el archivo volcado en nuestro disco duro, pero si lo ejecutamos no funcionara porque no tiene la IAT, ahora que estamos en el OEP la buscaremos, en este caso no es difícil nos situamos sobre el call a ***GetModuleHandleA*** y le damos a ***Follow in Dump*** para caer en plena tabla, desde aquí buscamos el inicio y el final mirando arriba y abajo, esto no será problema asi que os doy esos datos directamente:

IAT => Inicio: 961000
Final: 965818
Largo: 4818

Si vamos mirando encontramos el primer puntero redirigido en [96100c]=3db0000, si ponemos en este puntero inválido un ***BPHardware On Write dword*** y reiniciamos vemos que va rellenando la tabla aquí:

```
:0700B1E1    8906          MOV DWORD PTR DS:[ESI],EAX
```

en este momento EAX tiene la direccion de la API y lo escribe en la tabla ya que [ESI] representa precisamente la IAT, esta zona no nos interesa mucho porque no hay mágico a la vista, estamos en el típico Vbox al que hay que injertar para obtener una IAT decente, gracias a RNarvaja tenemos un injerto infalible en estos casos de destrucción de tabla, jeje... pues reparar manualmente 209 entradas es un poco difícil, asi que nada a injertar se ha dicho, primeramente necesitamos unos cuantos datos para la susodicha rutinilla, como son el inicio de la IAT que sabemos esta en :961000, el final de la misma 965818, tenemos que buscar un lugar libre del ejecutable y allí injertar, este lugar puede ser por ejemplo :960B00, vale ahora un apunte mas hay punteros validos que apuntan a librerías propias de programa que casualmente tiene cargadas en direcciones muy similares a las de ***vboxtb.dll*** y esto nos dará problemas, para solventar esto le añadiremos unas líneas al injerto original de Ricardo, estas líneas tendrán la misión principal de evitar que se "cuelen" los punteros válidos que tienen direcciones similares a la de los punteros inválidos, lo haremos facilito, sabemos que en [96AFC] estará la direccion de la IAT del puntero que estamos tratando en cada una de las pasadas del bucle, entonces añadiremos 4 comparaciones para saber si estamos en alguno de los punteros llamémoslos conflictivos, de ser así cambiaremos el valor de [96AFC] y seguiremos a partir del lugar adecuado, antes de seguir delimitemos los puntos calientes, veamos un ejemplo:

Address	Hex dump	ASCII
00961000	BB 28 DA 77 B1 63 DB 77 10 24 DA 77 00 00 E0 03	╗(┌w▒c╫w►$┌w..σ♥
00961010	00 00 E1 03 99 4E DA 77 00 00 00 00 0E A6 45 02	..ß♥ÖN┌w....♫ªE☺
00961020	84 A4 45 02 00 00 00 00 03 4A 80 02 6E 4A 80 02	äñE☺....♥JÇ☺nJÇ☺
00961030	00 00 00 00 E8 6D 34 02 12 71 34 02 00 00 00 00	....Φm4☺↕q4☺....
00961040	7B 2F 64 02 0C 30 64 02 5C 30 64 02 D4 2D 64 02	{/d☺.0d☺\0d☺Ô-d☺
00961050	30 1F 64 02 19 2D 64 02 00 23 64 02 96 2D 64 02	0▼d☺↓-d☺.#d☺ü-d☺
00961060	E4 29 64 02 D8 18 64 02 77 43 64 02 9F 18 64 02	ä)d☺ïfd☺wCd☺ƒfd☺
00961070	16 43 64 02 31 36 65 02 1D BB 64 02 3E BC 64 02	▬Cd☺16e☺↔╗d☺>╝d☺

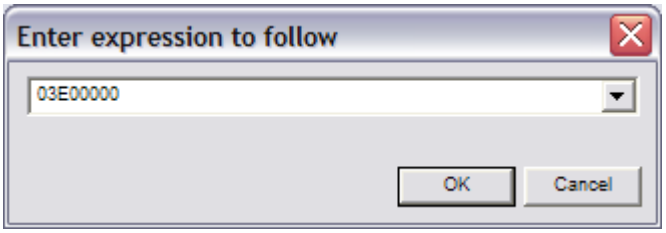
Allí se ve lo que os quiero explicar, fijaos en los tres primeros punteros que son del tipo 7xxxxxxx, luego el cuarto es del tipo 03E0xxxx (puntero inválido que debemos reparar), y poco después otros punteros tipo 0245xxxx, estos parecen inválidos porque las API's suelen estar para WinXP en 77xxxxxx como el primer caso, pero no! Estos punteros son correctos, si miramos en la ventana de memoria justo en la zona que marca veremos esto:

0240E000	0001D000 (118784.)	ACE	.rdata	imports,exp	Imag	R	RWE
0242B000	0000C000 (49152.)	ACE	.data	data	Imag	R	RWE
02437000	00001000 (4096.)	ACE	.rsrc	resources	Imag	R	RWE
02438000	00007000 (28672.)	ACE	.reloc	relocations	Imag	R	RWE
02440000	00001000 (4096.)	AGM		PE header	Imag	R	RWE
02441000	00115000 (1134592.	AGM	.text	code	Imag	R	RWE
02556000	00032000 (204800.)	AGM	.rdata	imports,exp	Imag	R	RWE
02588000	00041000 (266240.)	AGM	.data	data	Imag	R	RWE
025C9000	00001000 (4096.)	AGM	.rsrc	resources	Imag	R	RWE
025CA000	00017000 (94208.)	AGM	.reloc	relocations	Imag	R	RWE
025F0000	00001000 (4096.)	CoolType		PE header	Imag	R	RWE
025F1000	00105000 (1069056.	CoolType	.text	code	Imag	R	RWE
026F6000	0002A000 (172032.)	CoolType	.rdata	imports,exp	Imag	R	RWE
02720000	00054000 (344064.)	CoolType	.data	data	Imag	R	RWE
02774000	00001000 (4096.)	CoolType	.rsrc	resources	Imag	R	RWE
02775000	00017000 (94208.)	CoolType	.reloc	relocations	Imag	R	RWE
02790000	00001000 (4096.)	ARE		PE header	Imag	R	RWE

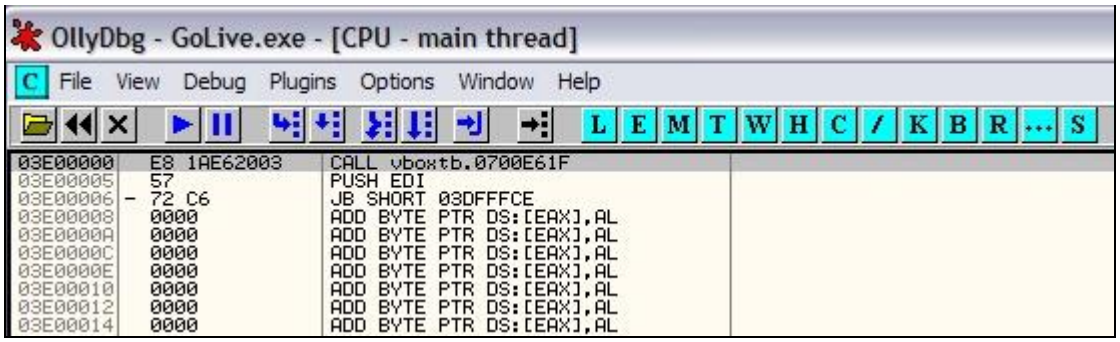
Allí vemos que hay una dll cargada en esa dirección, más concretamente su sección ***.text*** es la que alberga la dirección que marca el puntero de la IAT, así que estas entradas no las debe procesar nuestra rutina IMPORTANTE ESTE DETALLE!! Ahora sigamos, centrémonos en esta imagen, mirad el puntero seleccionado:

```
Address   Hex dump                                          ASCII
00961000  BB 28 DA 77 B1 63 DB 77 10 24 DA 77 00 00 E0 03  ╗(┌w▒c█w►$┌w..α♥
00961010  00 00 E1 03 99 4E DA 77 00 00 00 00 0E A6 45 02  ..ß♥ÖN┌w....♫ªE☻
00961020  84 A4 45 02 00 00 00 00 03 4A 80 02 6E 4A 80 02  ä¤E☻....♥JÇ☻nJÇ☻
00961030  00 00 00 00 E8 6D 34 02 12 71 34 02 00 00 00 00  ....Φm4☻↕q4☻....
00961040  7B 2F 64 02 0C 30 64 02 5C 30 64 02 D4 2D 64 02  {/d☻.0d☻\0d☻╘-d☻
00961050  30 1F 64 02 19 2D 64 02 00 23 64 02 96 2D 64 02  0▼d☻↓-d☻.#d☻ü-d☻
00961060  E4 29 64 02 D8 18 64 02 77 43 64 02 9F 18 64 02  ♂)d☻╪↑d☻wCd☻ƒ↑d☻
00961070  16 43 64 02 31 36 65 02 1D BB 64 02 3E BC 64 02  ▬Cd☻16e☻↔╗d☻>╝d☻
00961080  D7 25 64 02 4F 17 64 02 26 40 64 02 C5 2A 64 02  ╫%d☻O↨d☻&@d☻┼*d☻
00961090  0E 2B 64 02 A1 1C 64 02 4E 2A 64 02 7A 1A 64 02  ♫+d☻¡∟d☻N*d☻z→d☻
```

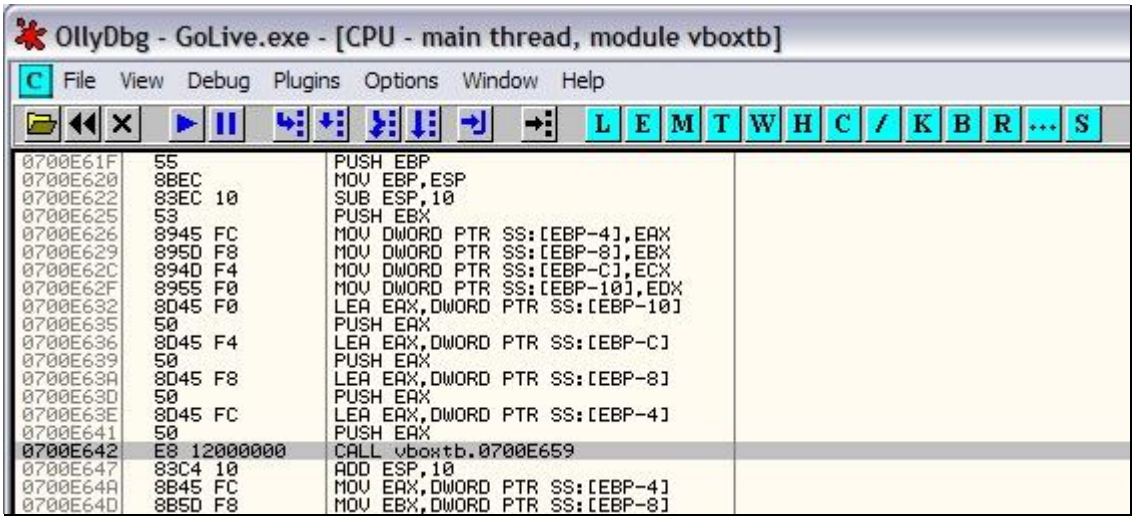
El valor que marca es “00 00 E0 03” que como sabemos se lee al revés, o sea “ 03 E0 00 00” vamos hacia allí, pulsamos ***Ctrl+G*** e introducimos el valor, asi:



Pulsamos en el botón ***OK*** y caemos en esa zona:



Allí vemos un salto a vboxtb.dll concretamente a :0700E61F, veamos que hay alli:



si seguimos los demás punteros redirigidos de la tabla veremos que todos al final apuntan a esta rutina indirectamente, o sea que aquí se decide que ***API*** corresponde a cada uno de ellos en función de la posición desde donde se llama hace los cálculos pertinentes y retorna la API, jeje.. le hemos pillado, mas concretamente a la salida del ***call 0700E569*** tenemos en EAX la dirección de la API, explico todo esto para que se entienda el injerto, ahora miramos la IAT y anotamos los rangos de direcciones que no debemos procesar, estos son:

96101C-9610C0 **/*Intervalo que no debe ser procesado…**
961804-962C6C **/*Intervalo que no debe ser procesado…**
963078-96554C **/*Intervalo que no debe ser procesado…**

965560-965648 **/*Intervalo que no debe ser procesado…**
96579C-9657DC **/*Intervalo que no debe ser procesado…**

Evitaremos pasar por estos tramos de la IAT para no caer en error, solamente nos quedará añadir las comprobaciones al injerto para que detecte cuando estemos en uno de estos casos y actúe en consecuencia, por ejemplo de esta forma:

Injerto:

```
:00960B00   C705 FC0A9600 0>  MOV DWORD PTR DS:[960AFC],GoLive.00961000     ;Inicio IAT en [960AFC]
:00960B0A   A1 FC0A9600       MOV EAX,DWORD PTR DS:[960AFC]                 ;obtenemos direccion
:00960B0F   3D 1C109600       CMP EAX,GoLive.0096101C                       ;es el caso 1?
:00960B14   74 4C             JE SHORT GoLive.00960B62                      ;tratamos caso1
:00960B16   3D 04189600       CMP EAX,GoLive.00961804                       ;es el caso 2?
:00960B1B   74 51             JE SHORT GoLive.00960B6E                      ;tratamos caso2
:00960B1D   3D 78309600       CMP EAX,GoLive.00963078                       ;es el caso 3?
:00960B22   74 56             JE SHORT GoLive.00960B7A                      ;tratamos caso3
:00960B24   3D 60559600       CMP EAX,GoLive.00965560                       ;es el caso 4?
:00960B29   74 5B             JE SHORT GoLive.00960B86                      ;tratamos caso4
:00960B2B   3D 9C579600       CMP EAX,GoLive.0096579C                       ;es el caso 5?
:00960B30   74 63             JE SHORT GoLive.00960B95                      ;tratamos caso5
:00960B32   8B00              MOV EAX,DWORD PTR DS:[EAX]                    ;obtiene direccion de la
:00960B34   3D 00000001       CMP EAX,GoLive.01000000                       ;posible API en EAX
:00960B39   7C 11             JL SHORT GoLive.00960B4C                      ;salta si no hay que reparar
:00960B3B   3D 00000004       CMP EAX,4000000                               ;hay que procesar puntero?
:00960B40   77 0A             JA SHORT GoLive.00960B4C                      ;salta sin procesarlo
:00960B42   FFD0              CALL EAX                                      ;Obtenemos direccion API
:00960B44   8B1D FC0A9600     MOV EBX,DWORD PTR DS:[960AFC]                 ;En EBX direccion IAT
:00960B4A   8903              MOV DWORD PTR DS:[EBX],EAX                    ;Guardamos el puntero
:00960B4C   8305 FC0A9600 04  ADD DWORD PTR DS:[960AFC],4                   ;apuntamos al séte dword
:00960B53   813D FC0A9600 1>  CMP DWORD PTR DS:[960AFC],GoLive.00965818     ;hemos terminado?
:00960B5D 74 02               JE SHORT GoLive.00960B61                      ;SI => Salimos...
:00960B5F   EB A9             JMP SHORT GoLive.00960B0A                     ;Seguimos trabajando
:00960B61   90                NOP    <== BPX Aquí para parar cuando termine el trabajo…!!!
:00960B62   C705 FC0A9600 C>MOV DWORD PTR DS:[960AFC],GoLive.009610C0       ;tratamos caso1
:00960B6C   EB 9C             JMP SHORT GoLive.00960B0A
:00960B6E   C705 FC0A9600 6>  MOV DWORD PTR DS:[960AFC],GoLive.00962C6C     ;tratamos caso2
:00960B78   EB 90             JMP SHORT GoLive.00960B0A
:00960B7A   C705 FC0A9600 4>  MOV DWORD PTR DS:[960AFC],GoLive.0096554C     ;tratamos caso3
:00960B84   EB 84             JMP SHORT GoLive.00960B0A
:00960B86   C705 FC0A9600 4>  MOV DWORD PTR DS:[960AFC],GoLive.00965648     ;tratamos caso4
:00960B90   E9 75FFFFFF       JMP GoLive.00960B0A
:00960B95   C705 FC0A9600 D>  MOV DWORD PTR DS:[960AFC],GoLive.009657DC     ;tratamos caso5
:00960B9F   E9 66FFFFFF       JMP GoLive.00960B0A
:00960BA4   90                NOP
```

En binario:

```
C7 05 FC 0A 96 00 00 10 96 00 A1 FC 0A 96 00 3D 1C 10 96 00 74 4C 3D 04 18 96 00 74 51 3D 78 30
96 00 74 56 3D 60 55 96 00 74 5B 3D 9C 57 96 00 74 63 8B 00 3D 00 00 00 01 7C 11 3D 00 00 00 04
77 0A FF D0 8B 1D FC 0A 96 00 89 03 83 05 FC 0A 96 00 04 81 3D FC 0A 96 00 18 58 96 00 74 02 EB
A9 90 C7 05 FC 0A 96 00 C0 10 96 00 EB 9C C7 05 FC 0A 96 00 6C 2C 96 00 EB 90 C7 05 FC 0A 96 00
4C 55 96 00 EB 84 C7 05 FC 0A 96 00 48 56 96 00 E9 75 FF FF FF C7 05 FC 0A 96 00 DC 57 96 00 E9
66 FF FF FF 90
```

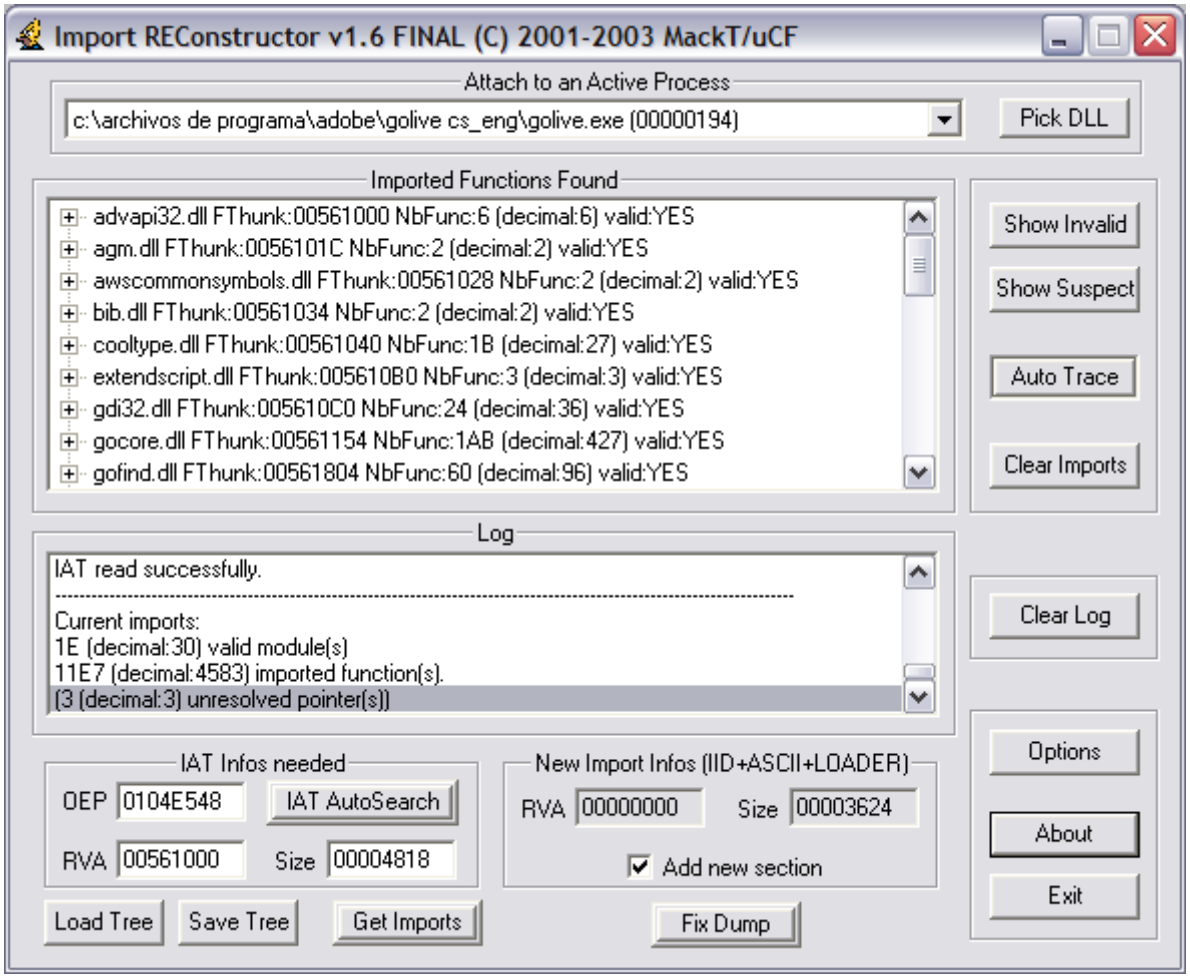
Este injerto irá en ***GoLive.exe*** en la dirección :960B00 ahora hay que injertar también el ***vboxtb.dll*** para que funcione todo, allí EAX tomará la dirección de la API correcta entonces forzaremos un salto hacia nuestra rutina para guardar este valor en la entrada de la IAT que estemos procesando en ese momento, bien sencillo, injertaremos estas dos líneas de código:

```
:0700E647   68 3D0B9600       PUSH 960B3D
:0700E64C   C3                RETN
```

Preparamos el injerto que deberá quedar de esta manera:

```
OllyDbg - GoLive.exe - [CPU - main thread, module GoLive]
00960B00   C705 FC0A9600 00109600   MOV DWORD PTR DS:[960AFC],GoLive.00961000
00960B0A   A1 FC0A9600              MOV EAX,DWORD PTR DS:[960AFC]
00960B0F   3D 1C109600              CMP EAX,GoLive.0096101C
00960B14 v 74 4C                    JE SHORT GoLive.00960B62
00960B16   3D 04189600              CMP EAX,GoLive.00961804
00960B1B v 74 51                    JE SHORT GoLive.00960B6E
00960B1D   3D 78309600              CMP EAX,GoLive.00963078
00960B22 v 74 56                    JE SHORT GoLive.00960B7A
00960B24   3D 60559600              CMP EAX,GoLive.00965560
00960B29 v 74 5B                    JE SHORT GoLive.00960B86
00960B2B   3D 9C579600              CMP EAX,GoLive.0096579C
00960B30 v 74 63                    JE SHORT GoLive.00960B95
00960B32   8B00                     MOV EAX,DWORD PTR DS:[EAX]
00960B34   3D 00000001              CMP EAX,GoLive.01000000
00960B39 v 7C 11                    JL SHORT GoLive.00960B4C
00960B3B   3D 00000004              CMP EAX,4000000
00960B40 v 77 0A                    JA SHORT GoLive.00960B4C
00960B42   FFD0                     CALL EAX
00960B44   8B1D FC0A9600            MOV EBX,DWORD PTR DS:[960AFC]
00960B4A   8903                     MOV DWORD PTR DS:[EBX],EAX
00960B4C   8305 FC0A9600 04         ADD DWORD PTR DS:[960AFC],4
00960B53   813D FC0A9600 18589600   CMP DWORD PTR DS:[960AFC],GoLive.00965818
00960B5D v 74 02                    JE SHORT GoLive.00960B61
00960B5F ^ EB A9                    JMP SHORT GoLive.00960B0A
00960B61   90                       NOP
00960B62   C705 FC0A9600 C0109600   MOV DWORD PTR DS:[960AFC],GoLive.009610C0
00960B6C ^ EB 9C                    JMP SHORT GoLive.00960B0A
00960B6E   C705 FC0A9600 6C2C9600   MOV DWORD PTR DS:[960AFC],GoLive.00962C6C
00960B78 ^ EB 90                    JMP SHORT GoLive.00960B0A
00960B7A   C705 FC0A9600 4C559600   MOV DWORD PTR DS:[960AFC],GoLive.0096554C
00960B84 ^ EB 84                    JMP SHORT GoLive.00960B0A
00960B86   C705 FC0A9600 48569600   MOV DWORD PTR DS:[960AFC],GoLive.00965648
00960B90 ^ E9 75FFFFFF              JMP GoLive.00960B0A
00960B95   C705 FC0A9600 DC579600   MOV DWORD PTR DS:[960AFC],GoLive.009657DC
00960B9F ^ E9 66FFFFFF              JMP GoLive.00960B0A
00960BA4   90                       NOP
```

Cuando lo tengamos listo le damos a ***RUN*** con ***F9*** y no pasara ni un segundo que Olly romperá justo sobre el NOP donde pusimos el ***BPX***, en este momento ya tenemos la tabla casi reparada, miremos el Import:



Quedaron tres entradas por resolver, las enumeraremos una a una y daremos la solución:

Entrada1:

IAT rva: 565770
Apunta a: 700EC12

```
DISASSEMBLER / HEX VIEWER

0700EC12    push ebp
0700EC13    mov ebp,esp
0700EC15    call 0700EBC1
0700EC1A    push dword ptr [ebp+18]
0700EC1D    push dword ptr [ebp+14]
0700EC20    push dword ptr [ebp+10]
0700EC23    push dword ptr [ebp+C]
0700EC26    push dword ptr [ebp+8]
0700EC29    call [704C3D4]    // = user32.dll/01FE/PeekMessageA
0700EC2F    pop ebp
0700EC30    retn 14
```

API correcta: PeekMessageA

Entrada2:

IAT rva: 5657C8
Apunta a: 03D90000

/* Este lo debería haber reparado el injerto, no se que habrá pasado, igualmente lo repararemos, un truquito: vamos al injerto y nos situamos al inicio con ***New origin here***, luego bajamos una instrucción hasta:

:00960B0A A1 FC0A9600 MOV EAX,DWORD PTR DS:[960AFC]

Aqui hacemos Follow in dump/Memory Address y cambiamos de:

```
Address   Hex dump                                          ASCII
00960AFC  00 10 96 00 C7 05 FC 0A 96 00 00 10 96 00 A1 FC  .►ü.Æ♣ª.ü..►ü.í ª
00960B0C  0A 96 00 3D 1C 10 96 00 74 4C 3D 04 18 96 00 74  .ü.=└►ü.tL=♦↑ü.t
00960B1C  51 3D 78 30 96 00 74 56 3D 60 55 96 00 74 5B 3D  Q=x0ü.tV='Uü.t[=
00960B2C  9C 57 96 00 74 63 8B 00 3D 00 00 00 01 7C 11 3D  £Wü.tcï.=...☺¦◄=
00960B3C  00 00 00 04 77 0A FF D0 8B 1D FC 0A 96 00 89 03  ...♦w. ïï#ª.ü.ë♥
```

a:

```
Address   Hex dump                                          ASCII
00960AFC  C8 57 96 00 C7 05 FC 0A 96 00 00 10 96 00 A1 FC  ╚Wü.Æ♣ª.ü..►ü.í ª
00960B0C  0A 96 00 3D 1C 10 96 00 74 4C 3D 04 18 96 00 74  .ü.=└►ü.tL=♦↑ü.t
00960B1C  51 3D 78 30 96 00 74 56 3D 60 55 96 00 74 5B 3D  Q=x0ü.tV='Uü.t[=
00960B2C  9C 57 96 00 74 63 8B 00 3D 00 00 00 01 7C 11 3D  £Wü.tcï.=...☺¦◄=
00960B3C  00 00 00 04 77 0A FF D0 8B 1D FC 0A 96 00 89 03  ...♦w. ïï#ª.ü.ë♥
```

Ahora seguimos traceando la rutina, al salir del ***CALL EAX*** miremos los registros:

```
Registers (3DNow!)
EAX 76366FBF comdlg32.PrintDlgA
ECX 0012FFE0
EDX 03610608
EBX 009657FC GoLive.009657FC
ESP 0012ED60
EBP 0012ED84
ESI 01C1902C GoLive.01C1902C
EDI 033700BE ASCII "®ü¿ü¾øÊêÏ¶ü¼ü£ïÊï¾¥"
```

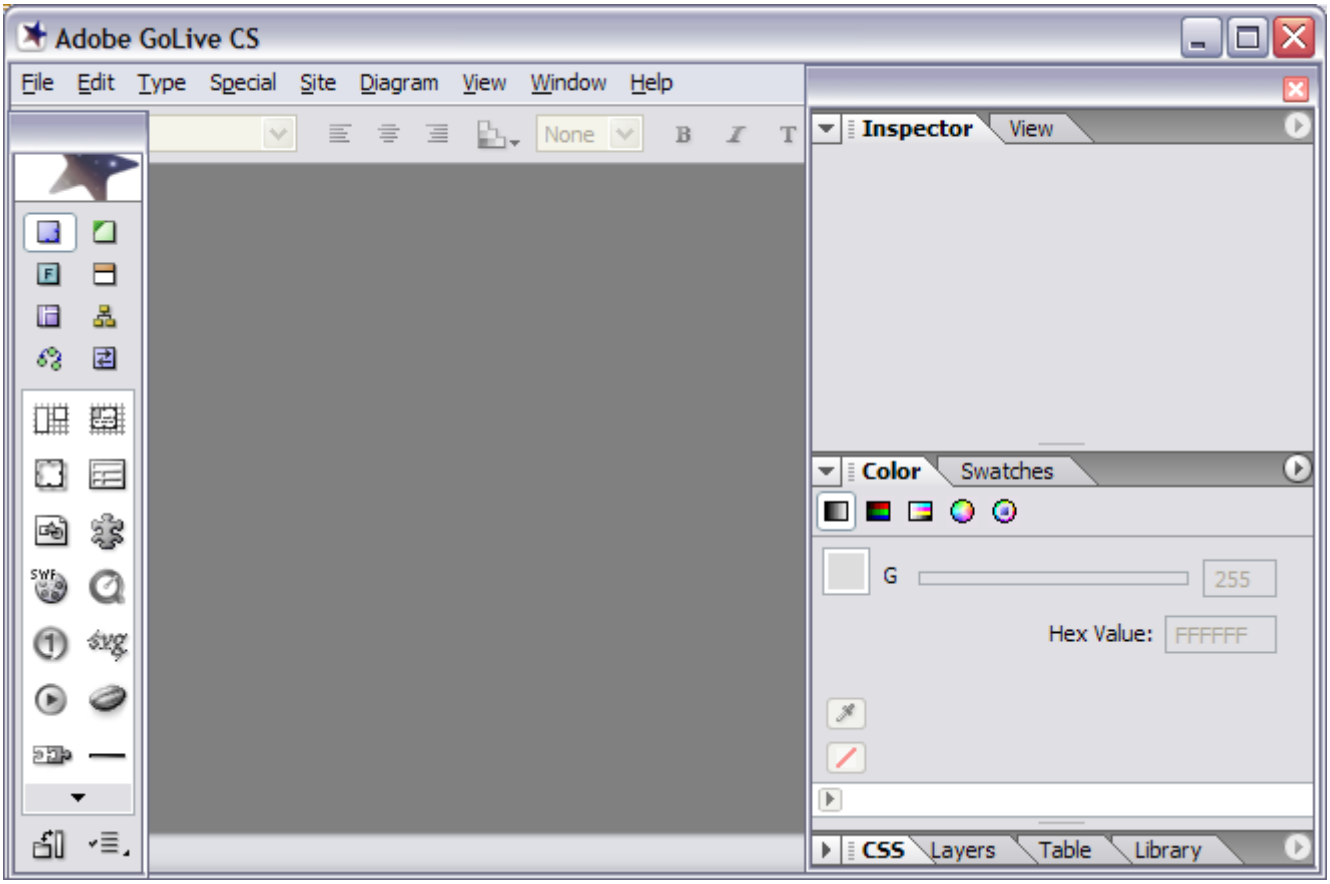
Allí tenemos el puntero que no se reparo antes, si ahora vamos al Import Reconstructor y pulsamos Get Imports ya la reconocerá como válida, entrada2 superada ***/***

Entrada3:

IAT rva: 5657D8
Apunta a: 03DA0000

/* Lo mismo que antes, al injerto y procesamos esta entrada que quedo sin reparar y esta poco después que la anterior, la reparamos totalmente igual e ImpRec la aceptara, trabajo resuelto*/

Ahora que hemos terminado os diré que estas dos ultimas entradas no se repararon porque calcule mal el ultimo intervalo ya que le di como final :9657DC cuando le debí haber dado :9657C8 y me hubiera evitado esto, pero tampoco eran muchas y no importa, ahora que tenemos la IAT resuelta damos a ***Fix Dump***, elegimos el dumpeado y probamos los resultados a ver que ocurre…



Pues ocurre que echa a correr pero que da gusto ;DDDDDDDDDDD…..

Como comentario final decir que no debemos preocuparnos por la limitación temporal ya que la llevaba el propio Vbox, ahora aparte de entrar sin ver la nagscreen inicial podemos utilizar el programa tanto tiempo como nos de la gana, jeje..

Y con esto damos por finalizado el tuto, espero que os haya gustado, pronto volveremos con otros, si alguien quiere contactar conmigo por cualquier tema, ya sea una duda, una sugerencia o como no una crítica siéntase libre de escribirme a cualquiera de estas direcciones:

syxe00@yahoo.es
syxe00@hotmail.com